

In-Class Lab 03 - Try, Catch, and Database Errors

Course: WEB-250

Time: About 15-20 minutes

Grading: Practice only; nothing to submit

Continue in: The same [web250/salamanders](#) folder

Before starting: Lab 02's table displays correctly. Locate the local PHP error log with the instructor.

Goal

Keep the same database summary, but handle a failed connection or SQL query deliberately. The visitor should see a clear message, while the developer can inspect technical details in the local PHP error log. This is the error-handling pattern used in asgn07.

We will not delete tables, stop a server shared with classmates, or change real data to produce a failure. A temporary wrong database name and a temporary misspelled table name are enough.

Step 1 - Read the control flow (about 3 minutes)

An **exception** is an object reporting that an operation could not finish normally. In Lab 01 we configured PDO with `PDO::ERRMODE_EXCEPTION`. When a query fails, PDO throws a `PDOException`; a failed PDO connection also throws one. [PDO error handling](#).

```
try
    Connect.
    Run the queries.
    Build the summary.

If all of that succeeds:
    Skip catch and continue to the HTML.

If a PDOException occurs:
    Skip the remaining statements in try.
    Run the matching catch block.
    Continue to the HTML after catch.
```

Catching an exception does not repair the cause, retry the failed query, or resume at the next statement inside `try`. Our page can continue because we supply a different output path. [PHP exceptions](#).

Step 2 - Put database work inside try (about 5 minutes)

Replace the **opening PHP block only** in `public/index.php` with the following. Keep the HTML for Step 3. The query statements are the ones from Lab 02, now enclosed by `try` and followed by `catch`.

```
<?php
require_once __DIR__ . '/../private/initialize.php';
```

```

// Set a known starting state before attempting database operations.
$database_name = '';
$summary = [];
$error_message = null;

try {
    $database = Database::connect();

    // query() returns a statement object. fetchColumn() retrieves one
    // value.
    $statement = $database->query('SELECT DATABASE()');
    $database_name = (string) $statement->fetchColumn();

    $statement = $database->query('SELECT COUNT(*) FROM salamanders');
    $salamander_count = (int) $statement->fetchColumn();

    $statement = $database->query('SELECT COUNT(*) FROM habitats');
    $habitat_count = (int) $statement->fetchColumn();

    $statement = $database->query('SELECT COUNT(*) FROM
salamander_habitat_links');
    $link_count = (int) $statement->fetchColumn();

    // The labels are fixed, but every count came from a SQL query.
    $summary = [
        ['table_name' => 'salamanders', 'row_count' => $salamander_count],
        ['table_name' => 'habitats', 'row_count' => $habitat_count],
        ['table_name' => 'salamander_habitat_links', 'row_count' =>
$link_count],
    ];
} catch (PDOException $exception) {
    // Set the status before sending any HTML to the browser.
    http_response_code(500);

    // Technical details go to the local PHP error log, not to visitors.
    error_log('Salamanders database error: ' . $exception->getMessage());

    $error_message = 'Database check failed. Check the local PHP error
log.';
}
?>

```

Read these details before moving on:

`catch (PDOException $exception)` accepts the PDO exception object into a variable. Calling `$exception->getMessage()` retrieves its technical message. We log that message, but do not echo it into the page.

`http_response_code(500)` tells the browser that this request had a server-side failure. It does not print an error page. Keep it before the HTML; sending output first can prevent a status/header change. [HTTP response codes in PHP](#).

`$error_message` begins as `null`, meaning we have not recorded an error. The catch block replaces it with a readable message. We do **not** initialize failed counts to zero: zero records and an unavailable database are different situations.

`initialize.php` still only loads definitions, and `Database.php` still only manages the connection. The connection is opened **inside the page's try block**, so a failure there can reach this catch block. Keep raw HTML and error-output decisions out of the connection class.

Step 3 - Choose success or error output (about 3 minutes)

Inside `<body>`, keep the heading. Replace everything **after the heading and before `</body>`** with:

```
<?php if ($error_message !== null): ?>
    <p><?= h($error_message) ?></p>
<?php else: ?>
    <p>Database: <?= h($database_name) ?></p>

    <table>
        <caption>Database record counts</caption>
        <thead>
            <tr>
                <th scope="col">Table</th>
                <th scope="col">Rows</th>
            </tr>
        </thead>
        <tbody>
            <?php foreach ($summary as $item): ?>
                <tr>
                    <th scope="row"><?= h($item['table_name']) ?></th>
                    <td><?= h((string) $item['row_count']) ?></td>
                </tr>
            <?php endforeach; ?>
        </tbody>
    </table>
<?php endif; ?>
```

This is the same table from Lab 02, enclosed by a condition. `if / else / endif` uses the same template-style syntax as `foreach / endforeach`.

Refresh **before** changing any settings. The normal result must still be `wnc_salamanders` and counts of **5, 3, and 9** with the supplied data.

On failure, the page will show the heading and generic message but no success table. The closing HTML still renders. The table is not replaced with misleading zeroes or partial counts.

Step 4 - Cause two controlled failures (about 5 minutes)

Test A - Connection failure

In `private/db_credentials.php`, temporarily change only:

```
const DB_NAME = 'wnc_salamanders_missing_lab';
```

Use a name that does not exist in your local database server. Refresh. Expect:

```
WNC Salamanders Database Check
Database check failed. Check the local PHP error log.
```

Check the newest entry in the PHP error log. It should include our prefix, **Salamanders database error:**, followed by a database connection error. A missing database often produces **Unknown database**; the exact message depends on server permissions and configuration.

Restore **DB_NAME** to **wnc_salamanders**. Refresh and verify the working table **before** Test B. PHP loads the restored constant in the new request.

Test B - Query failure after a successful connection

In **public/index.php**, temporarily change only this query:

```
$statement = $database->query('SELECT COUNT(*) FROM salamanders_typo');
```

Do not rename the real table. Refresh. The same browser message should appear, but the new log entry should identify the nonexistent table/query problem. The connection succeeded; a later operation inside **try** failed.

Restore the query to **SELECT COUNT(*) FROM salamanders**. Refresh. Finish with the normal table working again.

Finding the log

On MAMP for macOS, open **/Applications/MAMP/logs/php_error.log**. [MAMP log locations](#).

For XAMPP or a customized installation, use the log configured for **the PHP serving this browser page**. The instructor can inspect that PHP configuration; it may differ from terminal PHP. Do not assume the MySQL log is where PHP's **error_log()** message appears.

If the location is still unclear, temporarily place this **inside <body>, after the heading**, on the local page:

```
<p>Configured PHP error log: <?= h((string) ini_get('error_log')) ?></p>
```

Refresh and open the indicated path. An empty result means no file path is specified there; the message may go to the server's error logger, so ask the instructor to check Apache's error log. A configured value of **syslog** means it uses the system logger. Remove this diagnostic line after locating the log. [ini_get\(\)](#) and [error_log\(\)](#).

Keep logs private. Technical messages can contain account names, paths, schema details, and parts of queries. The page should not show those details or credentials.

Explain it to a partner and restore the working version (about 2 minutes)

Why can two different failures produce the same browser message but different log entries? In Test B, why does PHP not execute the remaining queries in `try`?

Confirm that the correct database name and table name are restored, the summary table works, and the temporary log-path output is removed. Keep the completed application; there is nothing to submit.

Important limit: `catch (PDOException ...)` handles PDO exceptions, not every programming error. A PHP syntax error, missing required file, or unrelated type error is not fixed by this catch block. This lab is not a complete production error-handling system.

Connection to asgn07

Transfer the same three choices: keep connection/query work inside `try`, record diagnostic details privately, and show either a successful summary or an error message. Leave the bird and salamander credentials separate. `.env` configuration, filtering/callbacks, and image work remain later topics.